

LeJudge: Natural-Language Constraints for Latent World-Model Planning

A pre-registered study on PushT with LeWorldModel and the Jev decision model

Abdel*

September 2026

Abstract

Latent world-model planners such as LeWorldModel (LeWM) choose actions by minimising one number: the distance between an imagined final latent and an encoded goal image. That cost has no way to say “but not through the middle”. We study LeJudge, a cost module that accepts such rules in plain English without a language model in the planning loop. Linear probes describe each imagined latent in a few words from a closed vocabulary; a decision model (Jev) answers one typed yes/no question per constraint and imagined step; code folds the probabilities into the cross-entropy-method cost. An oracle checker run on the same words, through the same code path, separates the judge’s errors from the planner’s and the world model’s. On PushT, judging alone, Jev reaches 0.94 AUROC and 0.88 accuracy on imagined rollouts and loses about one point on held-out paraphrases, where a keyword checker loses 18; it also fires on near-miss sentences about the wrong object or wall 0.38–0.53 of the time. In planning, from random starts no judge, the oracle included, changes the violation rate, because the goal image and the rule usually conflict. Where a compliant path exists, the oracle and keyword judges lower violations by 10 and 14 points on one set, short of the pre-registered 20, and Jev with its confidence gate disabled matches the oracle on another (8 points, $p_{\text{Holm}} = 0.039$). The remaining error comes from the world model. With a perfect judge, the chosen plan is imagined to break the spatial rule in 43% of episodes and breaks it in 92% when executed. All nine pre-registered hypotheses are scored against their thresholds; six are met. The 7,885 judge responses are committed and the paper rebuilds offline.

1 Introduction

World models are becoming planners. JEPA-style models learn to predict future *representations* rather than pixels [3, 15], and a line of recent work plans directly in those representations: DINO-WM [28], PLDM [23], V-JEPA 2 [4] and LeWM [20], which trains end to end from pixels and plans PushT by sampling action sequences, rolling them forward in latent space, and scoring each by the squared distance between its imagined final latent and the encoded goal image. The recipe is small and fast (on our laptop GPU one LeWM replan takes 1.20s), and the planner touches exactly one scalar. That is also the weakness. A goal image says where things should end up; it cannot say how to get there, what to avoid on the way, or what not to touch yet. Everything the user wants besides “end up looking like this” has to be engineered into that scalar by someone who can write cost functions.

*Code, cache and traces: <https://github.com/AbdelStark/lejjudge-jev-jepa>. Every number in this paper is read from the committed results tables by `paper/fill.py`; the build makes no network call.

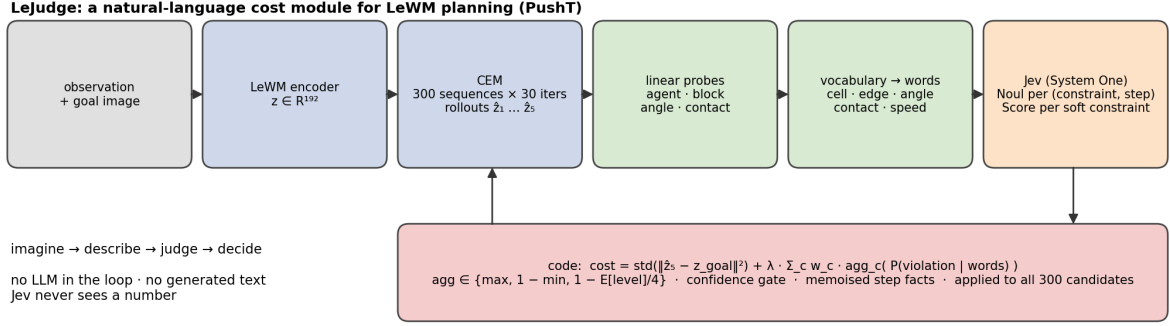


Figure 1: LeJudge. LeWM imagines latent futures for 300 candidate action sequences; linear probes describe each imagined latent in words; Jev answers one typed question per (constraint, step); code aggregates the probabilities and adds them to the goal-distance cost of every candidate. Owner text and code-generated facts live in separate fields of the state, and Jev never sees a number.

LeCun’s architecture sketch puts a *configurable cost module* next to the world model [15]. We test a narrow, measurable version of that idea: can a non-expert configure the cost with a sentence, and can the sentence be enforced inside a sub-second model-predictive-control loop without a generative model in the loop? Our system, LeJudge, separates three things that LLM-based reward design usually fuses [18, 22, 26]: *describing* an imagined state (probes and a closed vocabulary), *judging* a sentence against the description (a decision model that returns probabilities for typed questions), and *deciding* (arithmetic in code). The separation is what makes each part measurable. Because the description is a handful of words, an oracle checker can judge exactly the same input as the model, and every gap between them is attributable (Figure 1).

We measure, with pre-registered designs and an oracle in the same code path, where each part of such a system helps and where it runs out. We do not claim to have solved language-constrained planning. The contributions are:

1. **A measured bridge from latents to words.** Linear probes on LeWM’s 192-d latent produce vocabulary words that match ground truth 0.91–0.99 of the time on encoded frames and 0.84–0.98 five imagined steps out (Section 5.1).
2. **A judge-only benchmark with an oracle.** Twelve constraints in three families, each with five held-out paraphrases and two near-miss negatives, scored on identical inputs by Jev, a keyword checker, a local LLM and the oracle. Jev is accurate, reasonably calibrated and paraphrase-robust, and too willing to enforce a sentence that names the wrong object or wall (Section 5.2).
3. **Three pre-registered planning studies** that separate judge error, cost-mechanism error and world-model error: a null result explained by the task, a flaw in the original cost found with the oracle and fixed before any Jev planning result, a direct measurement of the gap between imagined and executed violations, and a scoring of every hypothesis against its threshold (Sections 5.3–5.8).
4. **An offline, fully cached build:** responses, traces, decisions and every deviation from the pre-registration are committed (Section 8).

2 Related work

Planning in latent space. DINO-WM [28] plans with frozen DINOv2 features and CEM; PLDM [23] argues for latent dynamics learned from reward-free data; TD-MPC2 [12] learns a latent model jointly with a value function; V-JEPA 2 [4] plans robot actions towards goal images with an action-conditioned video predictor. LeWM [20] trains encoder and predictor end to end from pixels and ships in `stable-worldmodel` [19], whose pluggable per-candidate objective we build on. All of these specify the task with a goal; constraints, when present, are hand-coded costs.

Constraints in control. Constrained MPC and safe RL encode rules as inequality constraints or penalties on state [1, 9], written by an engineer. We keep the penalty formulation and replace the hand-written function with a judged sentence, which is why the open questions in this paper are about the judge and the model, not the optimiser.

Language to cost. One family translates language once into an executable artefact: Eureka [18] and Language to Rewards [26] have an LLM write reward code or reward parameters for an optimiser or MPC; Code as Policies [16] and VoxPoser [14] compose language into programs or 3-D value maps for a motion planner; Lang2LTL [17] grounds commands into temporal logic. A compiled artefact is fast and auditable once written, but covers only what its grammar or code can express, and a wrong translation fails silently. A second family scores at run time with a generative model: VLM-RM [22] uses vision-language similarity as reward, and LLM-as-a-judge [27] grades text. LeJudge sits between the two. It judges the sentence afresh at run time, so wording is open, but the input is a closed-vocabulary description and the output a probability for a typed question. That keeps calls small (about 5.5 calls and 31,201 input tokens per episode with memoisation), lets an oracle score the same inputs, and keeps untrusted owner text in a separate field from machine-generated facts, the separation that indirect prompt injection exploits when it is missing [10].

Probing. Linear probes are the standard tool for asking what a representation encodes [2]; Maes et al. [20] report that PushT positions and angles are linearly recoverable from LeWM latents. We use probes as a component with a measured error budget, not as an analysis.

3 Method

3.1 Setting and notation

An episode starts from an observation o_0 with goal image o_g . LeWM’s encoder maps images to $z \in \mathbb{R}^{192}$; its predictor rolls a latent forward under a block of actions. Following the authors’ PushT protocol, one planning step is a block of 5 environment steps, the horizon is $H = 5$ blocks, and the policy executes all 5 blocks before replanning. The CEM samples $S = 300$ action sequences per iteration for 30 iterations and refits on the 30 lowest-cost candidates [6]. For candidate k the imagined latents are $\hat{z}_{1..H}^{(k)}$ and the base cost is $A_k = \|\hat{z}_H^{(k)} - z_g\|^2$. A constraint set $\mathcal{C}$ is a list of sentences c , each with a family (**never**, **always**, **soft**) and a weight w_c .

3.2 Probes and vocabulary

A ridge-regression head maps z to the agent position, the block position, $(\sin \theta, \cos \theta)$ of the block angle and a contact logit. It is trained on 93,536 frames from 800 expert episodes encoded with

the released checkpoint, split by episode. A deterministic vocabulary (`pusht@1`) turns these seven numbers into six words per step: the block’s and the agent’s cell on a 3×3 grid; whether the block polygon is within 10 px of a wall (*left/right/top/bottom edge* or *none*); one of six 60° angle bins (*upright, tilted left/right, on its side left/right, upside down*); *contact*, omitted when the logit is within ± 0.5 of zero; and *block speed* (*still, slow, fast*; thresholds 2 and 15 px per block, calibrated on expert data). The mapping is pure, and a unit test asserts that no digit ever reaches a fact.

3.3 Constraint library

Twelve PushT constraints were written before any result (Appendix A): eight **never** rules (e.g. “*Keep the T out of the centre cell.*”), three **always** rules (e.g. “*Approach the block from below.*”) and one **soft** rule (“*Be gentle: move the block slowly.*”). Each has an oracle checker over ground-truth state, five paraphrases written to preserve meaning, and two near-miss negatives that a careless matcher would confuse (for the centre rule, “*Keep the agent out of the centre cell.*”). The library is frozen and hashed (`61d416fdee6c`).

3.4 The judge call

One `system_one` call carries a JSON state with two top-level fields, `constraints` (owner text, keyed `c1..cC`) and `candidates` (code-generated facts, keyed `k1..kK`, one row per step), plus a dictionary of typed questions. For the hard families each question is a Noul, Jev’s yes/no primitive, which returns $P(\text{true})$ [24]: “*At step t of candidate k , does the block or agent violate constraint c exactly as written in `constraints.c`?*” Its criteria name the six fact fields and say what to answer when the facts do not mention what the constraint needs. For **soft** constraints the question is a five-level Score (*not at all ... fully*) per candidate. Templates are in Appendix B. They follow the model’s published guidance [24]: literal wording, one judgment per question, true means present, no counting or comparison, and separate fields for trusted and untrusted text. Aggregation is code: $\max_t p$ for **never**, $1 - \min_t p$ for **always**, $1 - \mathbb{E}[\text{level}]/4$ for **soft**.

3.5 Cost

At every judged CEM iteration the objective returns, for each candidate,

$$\text{cost}_k = \frac{A_k - \bar{A}}{\sigma_A} + \lambda \sum_{c \in \mathcal{C}} w_c \text{penalty}_c(k),$$

with A standardised over the population so that λ is unitless; $\lambda = 0$ is bit-identical to LeWM (regression test). A confidence gate *holds* a candidate, setting its penalty to zero, when more than a fraction τ of its per-step probabilities fall in $[0.3, 0.7]$ (for Score, when the model’s confidence is below $1 - \tau$). A held candidate is therefore scored exactly like a confidently compliant one; Section 6 returns to this.

Which candidates are judged. The original design (RFC-0004) judged only the $K = 16$ lowest- A candidates on the last three of 30 CEM iterations, on the reasoning that A already ranks. Section 5.3 shows, with a perfect judge, that this cannot change plans, so we judge *every* candidate. This is affordable because a hard-family question depends only on one step’s words and its index: the population’s $S \times H = 1,500$ steps collapse to a few hundred unique (t, facts) keys, which are judged once and memoised across iterations, replans, episodes and runs (34,026 keys after all studies). Candidates inherit their steps’ probabilities. Soft constraints, whose question is per candidate, are judged on the $K = 16$ lowest-cost candidates, and the others receive the shortlist’s mean penalty as

a prior, so that no candidate is preferred merely for being unjudged. CEM iterations 0, 5, $\dots$, 25 and the last are judged. This change was logged before any Jev result on the study sets (Appendix C).

3.6 Baselines

All judges implement one interface and receive byte-identical states and question keys. **Oracle-on-probes** runs each constraint’s ground-truth checker on the probe outputs. On items described with ground-truth words it runs on the ground truth itself and is exact by construction, except on the soft rule, whose frozen episode label and graded score use different thresholds (**gentle**; see the decision log). It is the upper bound for any judge of the given words. **Keyword** is a regex dictionary written from the canonical sentences only and applied verbatim to paraphrases. **Local LLM** sends the same JSON and question keys to Qwen2.5-7B-Instruct through an OpenAI-compatible endpoint and asks for a JSON object of probabilities. A hosted-LLM comparison was not funded for this build.

4 Experimental setup

Environment, data and protocol. `swm/PushT-v1` [5, 8, 19], the released LeWM PushT checkpoint (revision 22b330c28c27), and the authors’ evaluation protocol: start states are frames of held-out expert episodes, the goal is the frame 25 environment steps later, the budget is 50 steps, success is the environment’s pose criterion, and the episode ends at success. Expert states and actions come from the official dataset; the action standardiser is fitted on all readable expert actions.

Reproduction gate. Unconstrained LeWM reaches 86% success on 50 episodes with the paper’s three-frame context (90% with one). The LeWM paper reports PushT success only in bar charts, so this checks protocol fidelity rather than matching a quoted number. Our first attempt, with random-push data we collected ourselves, reached 6%. The model was trained on expert motion (action standard deviation about 0.2 against 0.5 in our data), and a CEM fed the wrong action scale exploits the model. We report this because it is the kind of silent failure that makes planning results hard to reproduce.

Constraint sets. The pre-registered sets **edges** and **edges+upright** were replaced before any study run. Under the expert-start protocol the block almost never reaches a side wall (0 of the 50 reproduction episodes) and starts upright in only 18% of Study 1 episodes, too rarely to be righted within the budget. The study sets are **spatial** = {centre_avoid}, **spatial+temporal** = {centre_avoid, no_contact_first3} and **implicit** = {gentle, approach_below}, chosen from constraints whose base violation rate under unconstrained LeWM lay between 30% and 80%.

Conditions and metrics. {LeWM, oracle-on-probes, keyword, Jev} $\times$ 3 sets $\times$ seeds {0,1,2} $\times$ 30 episodes per cell; $\lambda = 1$, $\tau = 0.5$. Conditions share start states, goals and CEM sampler seeds. The primary metrics are episode violation rate (the oracle checker on executed ground-truth states at the planner’s cadence) and success rate. We report 95% percentile-bootstrap intervals with 10,000 resamples [7], paired Wilcoxon signed-rank tests against LeWM on per-episode outcomes [25] with Holm correction across the sets of a study [13], and risk differences with episode-paired bootstrap intervals. Expected calibration error (ECE) [11] uses 10 equal-mass bins; with 64–128 items per subset each bin holds 6–13 items, so ECE values are indicative. Studies 2 and 3 were each pre-registered after the previous study; the pre-registration file and every amendment are hashed

in the decision log [21]. Because each study was motivated by the last, the family-wise error rate across the sequence is larger than any single corrected p -value suggests, and we treat Study 3 as confirmatory only for its own hypotheses.

5 Results

5.1 Probes

On 9,425 held-out expert frames the linear probe reaches $R^2 = 0.992$ for block position, 0.977 for agent position and 0.953 for the angle pair (mean block error 6.4px, angle error 4.9°); contact AUROC is 0.988. What the judge sees is words, and word accuracy is: block cell 0.95, agent cell 0.91, angle bin 0.94, edge 0.99, contact 0.95. Rolling the predictor open-loop with the expert’s own actions from 400 starts, word accuracy at $h = 5$ falls to 0.88 (block cell), 0.84 (agent cell), 0.88 (angle), 0.98 (edge) and 0.93 (contact), and block error grows to 15.0px (Figure 6a). An MLP head improves the angle ($R^2 = 0.972$, bin accuracy 0.96) but not the cell (0.90). The pre-registration names the linear probe; swapping in the MLP is one of the ablations (Table 6).

5.2 Judge-only study

The study has 256 items: 64 executed expert windows, each described once with ground-truth words and once with probe words (128 items), and 128 imagined CEM candidates described with probe words and labelled by the oracle checker on the probe outputs. Every item is scored against all twelve constraints in eight wordings (canonical, five paraphrases, two near-miss negatives), i.e. 24,576 (item, constraint, wording) judgments for each of Jev, the keyword checker and the oracle (73,728 rows), plus 4,608 rows from two extra repeats on a 64-item consistency subset and 192 rows from the local-LLM subset, 78,528 rows in all. Table 1 and Figures 2–3 summarise.

Accuracy and calibration. On imagined candidates described by probe words, the inputs the planner actually judges, Jev scores 0.88 accuracy, 0.78 F1, 0.94 AUROC and ECE 0.108; on executed windows the figures are 0.86, 0.82, 0.94 and 0.083. Describing the same executed windows with ground-truth words instead of probe words moves accuracy only from 0.86 to 0.88: at this probe quality the judge’s own errors dominate the description’s. Three repeats on 768 question pairs give a mean per-question standard deviation of 0.013 (max 0.098).

Paraphrases and negatives. Jev’s accuracy on paraphrases is 0.87 (imagined) and 0.85 (executed), against 0.88 and 0.86 on the canonical text. The keyword checker falls from 0.89 to 0.74 on imagined items and from 0.88 to 0.64 on executed ones (pooled, 0.89 to 0.71: a drop of 18 points, where the pre-registered H3 asked for at least 20). The near-miss negatives expose a real weakness. Jev fires on a sentence that swaps the object or the wall (*“Never let the T touch the top edge”* when the block touches the left edge) 0.38–0.53 of the time, against 0.02 for the keyword checker. A rule aimed at the wrong target would be enforced between a third and a half of the time; that is not acceptable in deployment, and the fix belongs in the question bank (Section 6). Per constraint (Figure 2), Jev is at or above 0.96 on the edge, tilt, upside-down, agent-cell and corner rules, lower on `centre_avoid` (0.86) and `upright_always` (0.82), and weakest on `stay_left_half` (0.74), where “left half” does not align with the 3×3 grid words, and `approach_below` (0.49), which requires relating the agent cell, the block cell and the first contact across steps.

Table 1: Judge-only study. Accuracy, F1, AUROC and ECE (10 equal-mass bins) at threshold 0.5 on canonical texts; accuracy on the five paraphrases; false-positive rate on near-miss negatives among items where the canonical condition is present (undefined for the oracle, which ignores wording); wall-clock seconds per 1,000 judgments on live calls. The local LLM returned schema-invalid answers in 67% of its calls and was run on canonical texts only; its row scores a failed call as if every answer were 0 (no violation for **never** rules, a violation for **always** rules).

judge	items	n	acc.	F1	AUROC	ECE	acc. (paraphr.)	near-miss FPR	s / 1,000
LeJudge / Jev	imagined, probe words	128	0.88	0.78	0.94	0.108	0.87	0.38	2.6
LeJudge / Jev	executed, probe words	64	0.86	0.82	0.94	0.083	0.85	0.53	2.5
LeJudge / Jev	executed, GT words	64	0.88	0.85	0.96	0.113	0.86	0.53	2.5
keyword checker	imagined, probe words	128	0.89	0.80	0.89	0.089	0.74	0.02	0.0
keyword checker	executed, probe words	64	0.88	0.83	0.90	0.097	0.64	0.02	0.0
local LLM (7B)	executed, probe words	8	0.59	0.47	0.59	0.353	–	–	566.0
local LLM (7B)	executed, GT words	8	0.61	0.49	0.61	0.332	–	–	566.0
oracle-on-probes	imagined, probe words	128	0.99	0.97	1.00	0.003	0.99	–	0.0
oracle-on-probes	executed, probe words	64	0.97	0.96	1.00	0.013	0.97	–	0.0
oracle-on-probes	executed, GT words	64	0.97	0.96	1.00	0.013	0.97	–	0.0

Table 2: Study 1 (random expert starts; $\lambda = 1$, $\tau = 0.5$; 3 seeds $\times$ 30 episodes per cell). Episode violation and success rates with 95% bootstrap intervals. A $\star$ would mark a paired Wilcoxon test against LeWM significant after Holm correction across sets; no cell qualifies.

condition	spatial		spatial+temporal		implicit	
	violation $\downarrow$	success $\uparrow$	violation $\downarrow$	success $\uparrow$	violation $\downarrow$	success $\uparrow$
LeWM (unconstrained)	0.57 [0.47, 0.67]	0.84 [0.77, 0.91]	0.86 [0.78, 0.92]	0.84 [0.77, 0.91]	0.57 [0.47, 0.67]	0.84 [0.77, 0.91]
keyword checker	0.56 [0.46, 0.66]	0.81 [0.72, 0.89]	0.81 [0.72, 0.89]	0.84 [0.77, 0.91]	0.56 [0.46, 0.66]	0.82 [0.74, 0.90]
oracle-on-probes	0.56 [0.46, 0.66]	0.81 [0.72, 0.89]	0.83 [0.76, 0.91]	0.83 [0.76, 0.90]	0.52 [0.42, 0.62]	0.82 [0.73, 0.90]
LeJudge / Jev ($\tau=0.5$)	0.56 [0.46, 0.66]	0.84 [0.77, 0.91]	0.84 [0.77, 0.91]	0.84 [0.77, 0.91]	0.59 [0.49, 0.69]	0.84 [0.77, 0.91]

Cost. Jev takes 2.6s per 1,000 judgments, because one call judges hundreds of questions. The local 7B LLM was affordable only on 8 executed windows (16 items, each window described with ground-truth and with probe words). It takes 566s per 1,000 judgments, returned schema-valid JSON in one call of three, and reached accuracy 0.73 and AUROC 0.74 on the valid answers. It bounds this class of baseline from below; it does not characterise it.

5.3 Study 1: planning from random expert starts

Table 2 and Figure 4 show a null result. Every condition, the oracle included, sits within a few points of unconstrained LeWM on all three sets (0 of nine Holm-corrected tests significant), at unchanged success. The oracle is correct about the words it is given, so this is not a judge failure. Two facts explain it.

First, the task often forbids compliance. On the spatial set 42 of 90 episodes (47%) have the goal block *inside* the centre cell and 6 more start there, so in 53% of episodes success and compliance are mutually exclusive. Stratifying post hoc (not pre-registered) by whether the start and goal states allow compliance, LeWM violates 14% of the remaining 42 satisfiable spatial episodes, and every condition has the same violation rate there.

Second, the original cost could not have helped in any case. It judged the 16 lowest-cost candidates while the CEM refits on the top 30, so almost half of every refit set was never penalised, and it acted only on the last three of 30 iterations, after the sampler had converged. With the oracle as judge on the spatial set, this design reproduced LeWM’s violation outcome in 60 of 60 paired episodes (an archived partial run), which is what led to judging the whole population

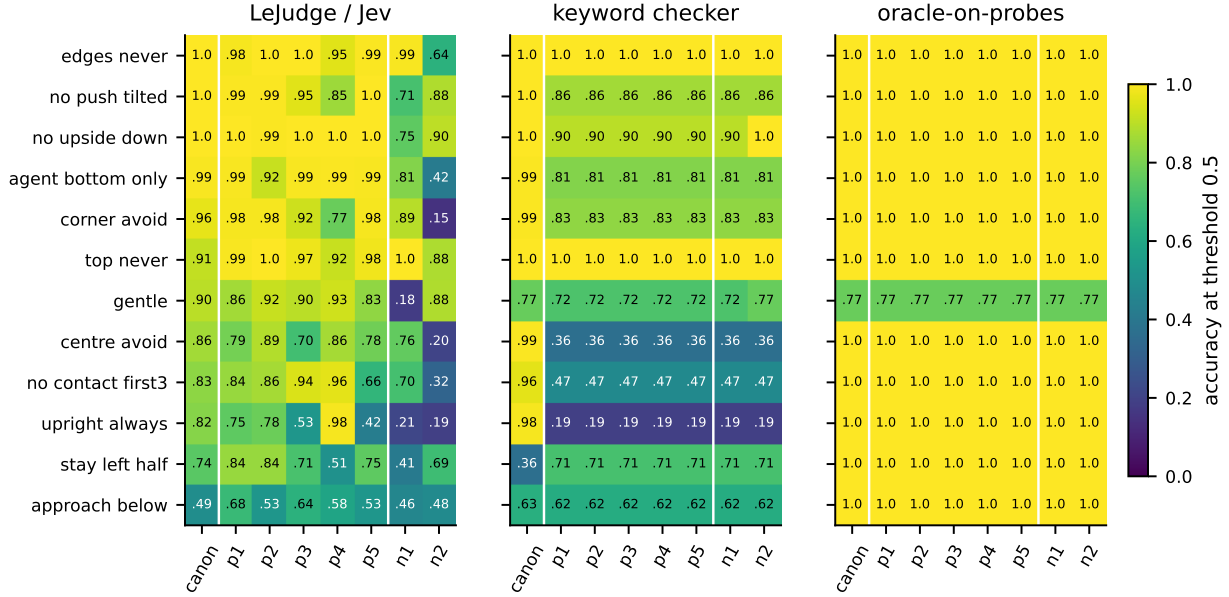


Figure 2: Accuracy per constraint and wording on probe-word items. Columns: canonical text, paraphrases p1–p5, near-miss negatives n1–n2. For negatives, accuracy is measured against the canonical label, so a low value means the judge fired on a sentence that does not apply. The keyword checker is written from the canonical sentence and does not transfer; the oracle ignores wording by construction.

Table 3: Study 2 (relevance-filtered starts; same conditions, seeds and budgets as Study 1). *: paired Wilcoxon vs LeWM significant after Holm correction across sets.

condition	spatial		spatial+temporal		implicit	
	violation ↓	success ↑	violation ↓	success ↑	violation ↓	success ↑
LeWM (unconstrained)	0.87 [0.79, 0.93]	0.92 [0.87, 0.97]	0.61 [0.51, 0.71]	0.64 [0.54, 0.74]	0.77 [0.68, 0.86]	0.71 [0.62, 0.80]
keyword checker	0.92 [0.87, 0.97]	0.92 [0.87, 0.98]	0.47* [0.37, 0.57]	0.68 [0.58, 0.77]	0.66* [0.56, 0.76]	0.71 [0.61, 0.80]
oracle-on-probes	0.92 [0.87, 0.97]	0.92 [0.87, 0.98]	0.51* [0.41, 0.61]	0.64 [0.54, 0.74]	0.69 [0.59, 0.79]	0.67 [0.57, 0.76]
LeJudge / Jev ($\tau=0.5$)	0.88 [0.81, 0.94]	0.93 [0.88, 0.98]	0.57 [0.47, 0.67]	0.72 [0.63, 0.81]	0.73 [0.63, 0.82]	0.73 [0.64, 0.82]

(Section 3.5). Study 1 was rerun with the amended cost, and the numbers above are from that rerun; the conclusion does not change, because the episodes themselves rarely admit compliant success.

5.4 Study 2: relevance-filtered starts

Study 2 keeps a start/goal window only when the constraint set is *satisfiable* from both end states and the expert’s own trajectory between them *violates* it, so that the task and the constraint are in tension and a compliant alternative is plausible. The filter uses dataset states only, is identical for every condition, and draws windows without replacement.

Table 3 and Figure 5 show a detectable but modest effect where a compliant path exists. On spatial+temporal the oracle lowers violations from 61% to 51% (risk difference -10 points, 95% CI $[-18, -2]$, $p_{\text{Holm}} = 0.038$) and the keyword checker to 47% (-14 points, $p_{\text{Holm}} = 0.001$). On implicit the keyword checker reaches 66% (-11 points, $p_{\text{Holm}} = 0.037$) and the oracle 69% (not significant). Pooled over the three sets, success is 76% for LeWM, 74% for the oracle, 77% for the keyword checker and 80% for Jev. These reductions fall short of the pre-registered 20 points

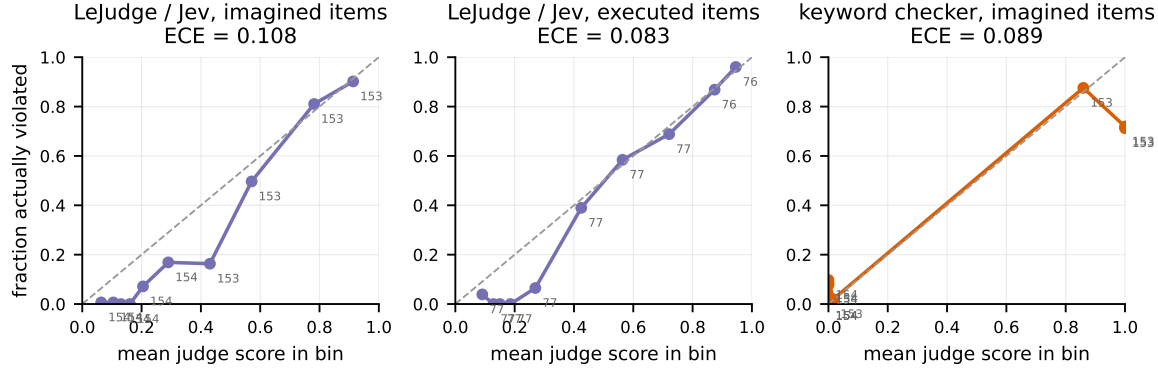


Figure 3: Reliability diagrams (10 equal-mass bins, canonical texts; bin sizes annotated). Jev’s scores rank well across the whole range, but between about 0.2 and 0.5 they overstate how often a violation is present (points below the diagonal). The keyword checker is binary, so its bins collapse onto two scores.

Table 4: Study 3: Jev with the confidence gate at $\tau \in \{0.5, 0.75, 1.0\}$ next to the Study 2 baselines on the same filtered episodes.

condition	spatial+temporal		implicit	
	violation ↓	success ↑	violation ↓	success ↑
LeWM (unconstrained)	0.61 [0.51, 0.71]	0.64 [0.54, 0.74]	0.77 [0.68, 0.86]	0.71 [0.62, 0.80]
keyword checker	0.47* [0.37, 0.57]	0.68 [0.58, 0.77]	0.66* [0.56, 0.76]	0.71 [0.61, 0.80]
oracle-on-probes	0.51* [0.41, 0.61]	0.64 [0.54, 0.74]	0.69 [0.59, 0.79]	0.67 [0.57, 0.76]
LeJudge / Jev ($\tau=0.5$)	0.57 [0.47, 0.67]	0.72 [0.63, 0.81]	0.73 [0.63, 0.82]	0.73 [0.64, 0.82]
Jev ($\tau=0.75$)	0.57 [0.47, 0.67]	0.68 [0.58, 0.78]	0.77 [0.68, 0.86]	0.66 [0.56, 0.76]
Jev, gate off ($\tau=1$)	0.56 [0.46, 0.66]	0.63 [0.53, 0.73]	0.69* [0.59, 0.78]	0.72 [0.63, 0.81]

for the oracle, so H1’ is not met; H2’, Jev within 10 points of the oracle on every set, is. Jev at the pre-registered $\tau = 0.5$ moves by -4 and -3 points, neither significant, while its gate held 23% and 21% of candidates on these sets (62% on spatial). On the spatial set nothing helps: violations are 87%–92% for every condition, and the λ sweep on these starts stays at 80%–90% (Figure 7). Section 5.6 shows why.

5.5 Study 3: the confidence gate

Study 3, pre-registered after Study 2, reruns Jev on the Study 2 episodes of the two responsive sets with $\tau \in \{0.75, 1.0\}$. With the gate disabled Jev reaches 69% violations on implicit, the oracle’s value, a reduction of 8 points against LeWM (CI $[-14, -2]$, $p_{\text{Holm}} = 0.039$), and 55.6% on spatial+temporal (-6 points vs LeWM, $p_{\text{Holm}} = 0.132$) against the oracle’s 51.1%. The pre-registered H1’’, gate-off Jev within five points of the oracle, holds on both sets (gaps 0.0 and 4.4 points). Success with the gate off is 72% on implicit (73% gated) and 63% on spatial+temporal (72% gated), costs of 1 and 9 points, inside the pre-registered 10-point tolerance (H2’’). The improvement over the gated variant is not itself significant at this sample size (-4 and -1 points, $p_{\text{Holm}} = 0.496$ and 0.739), and $\tau = 0.75$ does not differ from $\tau = 0.5$ ($p_{\text{Holm}} = 1.000$ and 0.731). Table 4 lists all cells.

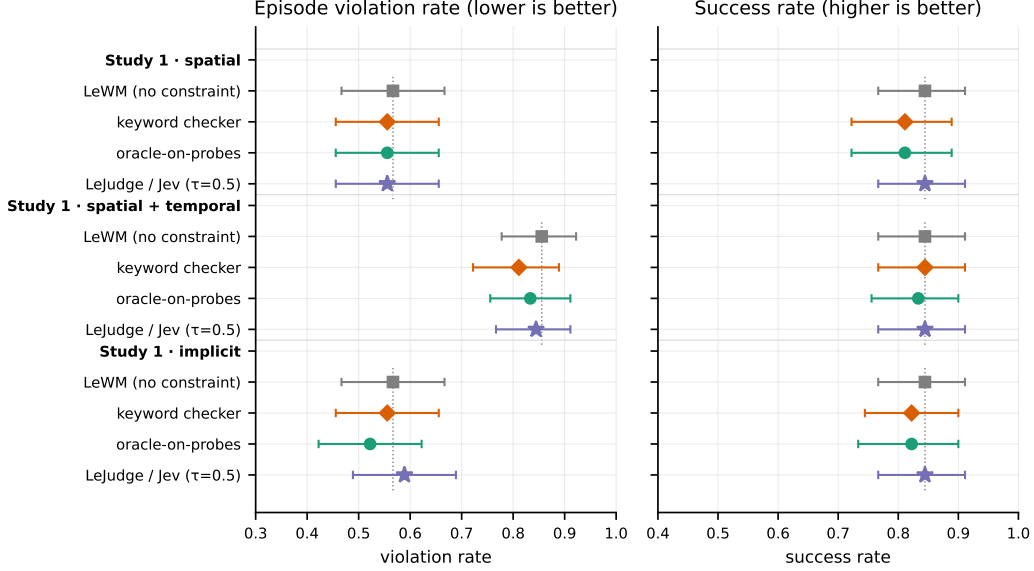


Figure 4: Study 1. Episode violation (left) and success (right) per condition and set, 95% bootstrap intervals; the dotted line marks LeWM.

Table 5: Imagined vs executed violation of the chosen plan (Study 2, first replan), and executed violation conditional on the imagined verdict. For Jev at $\tau = 0.5$ a held candidate carries zero penalty and counts as imagined-clean, so its “imagined” column is not comparable with the others.

set	judge	imagined	executed	executed imagined clean	executed imagined violating
spatial	oracle-on-probes	0.43	0.92	0.94	0.90
spatial	keyword checker	0.43	0.92	0.94	0.90
spatial	LeJudge / Jev	0.12	0.88	0.90	0.78
spatial+temporal	oracle-on-probes	0.36	0.51	0.36	0.75
spatial+temporal	keyword checker	0.37	0.47	0.33	0.69
spatial+temporal	LeJudge / Jev	0.64	0.57	0.52	0.61
implicit	oracle-on-probes	0.43	0.69	0.51	0.92
implicit	keyword checker	0.03	0.66	0.66	0.50
implicit	LeJudge / Jev	0.58	0.73	0.57	0.84

5.6 Where the residual comes from

Figure 6b and Table 5 read the traces. On the spatial set the oracle’s chosen plan is imagined to enter the centre in 43% of episodes but does so in 92% when executed, and conditional on an *imagined-clean* plan the executed violation rate is still 94%. The judge is right about the words it is given; the words describe a future that does not happen. On spatial+temporal, by contrast, an imagined-clean plan violates in 36% of executions and an imagined-violating one in 75%. There imagination is informative, the judge’s decisions carry through to execution, and that is where the effects of Studies 2 and 3 appear. Jev’s low imagined rate on spatial (12%) is the gate at work: it held 62% of candidates on that set, and a held candidate looks compliant.

We probed two planner-side fixes on the first 20 filtered spatial episodes of seed 0 (exploratory, not pre-registered). Executing the lowest-cost *sampled* candidate instead of the CEM mean, to avoid averaging paths that pass the centre on different sides, gives an oracle violation rate of 0.65 against 0.75 with the mean ($\lambda = 4$, every iteration judged). Replanning after every block instead of

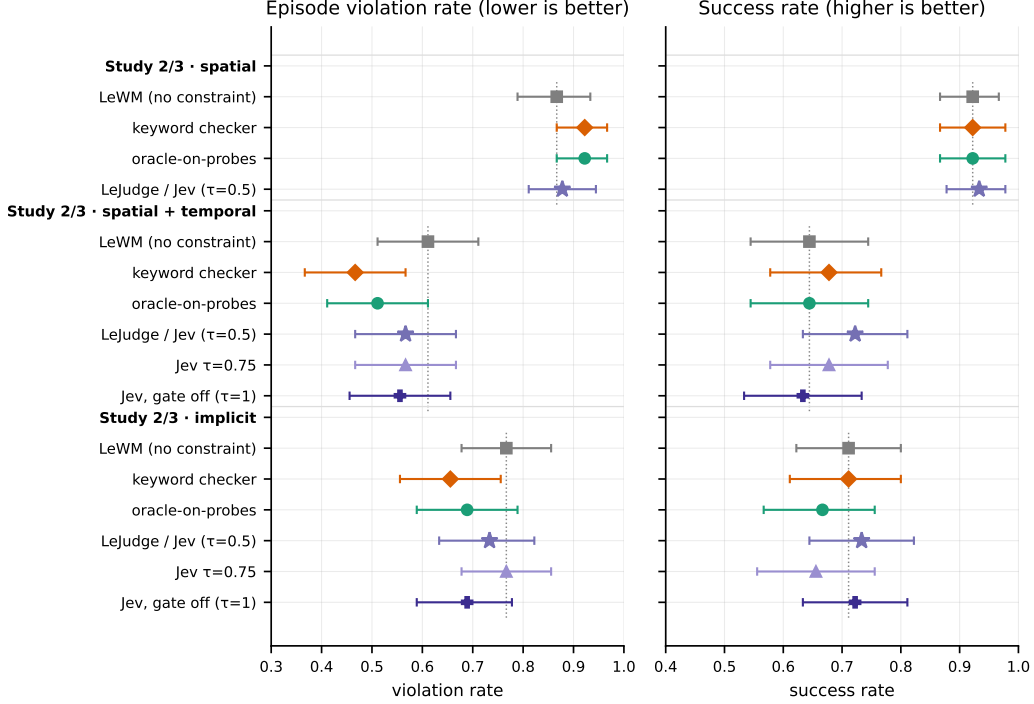


Figure 5: Studies 2 and 3 (relevance-filtered starts). Study 3 adds Jev with the gate relaxed ($\tau = 0.75$) and disabled ($\tau = 1$) on the same episodes.

every five gives unconstrained success of 30% against 95%. Neither is a fix at this budget. Figure 8 shows the population statistic the cost acts on: with the oracle as judge the penalised fraction rises over the first judged iterations and settles at 40%–46%, so the sampler keeps a mixed population rather than running out of compliant candidates.

5.7 Ablations

Table 6 varies λ , the soft-constraint shortlist K , the judging schedule, the prior for unjudged candidates, the vocabulary, the number of judged steps, hard rejection and the probe. It is a single-seed sweep with 20 episodes per cell, so it can only exclude large effects, and two groups of rows need reading with care. The K and prior rows cannot matter here, because the spatial set has only a **never** rule. The 4×4 vocabulary row does not test grid resolution: **pusht@2** has no cell called *centre*, so “Keep the T out of the centre cell” refers to nothing the words can express, and Jev’s gate never fires (held 0%). A rule whose referent the vocabulary lacks is therefore silently unenforceable, which is a finding in its own right. The remaining rows (λ , schedule, judged steps, hard rejection, MLP probe) vary things that could have mattered and change nothing detectable, as Section 5.6 predicts for this set. 8 of the 16 variants needed no new Jev call because the step-fact memo already held their words; the most expensive, judging every iteration, averaged 3.4 calls per episode.

5.8 Pre-registered hypotheses and outcomes

Table 7 scores every hypothesis in **docs/PREREG.md** against its threshold, with one-decimal values where a verdict depends on rounding. Six are met (H1, H2, H2', H3', H1'', H2''), one is not (H1'),

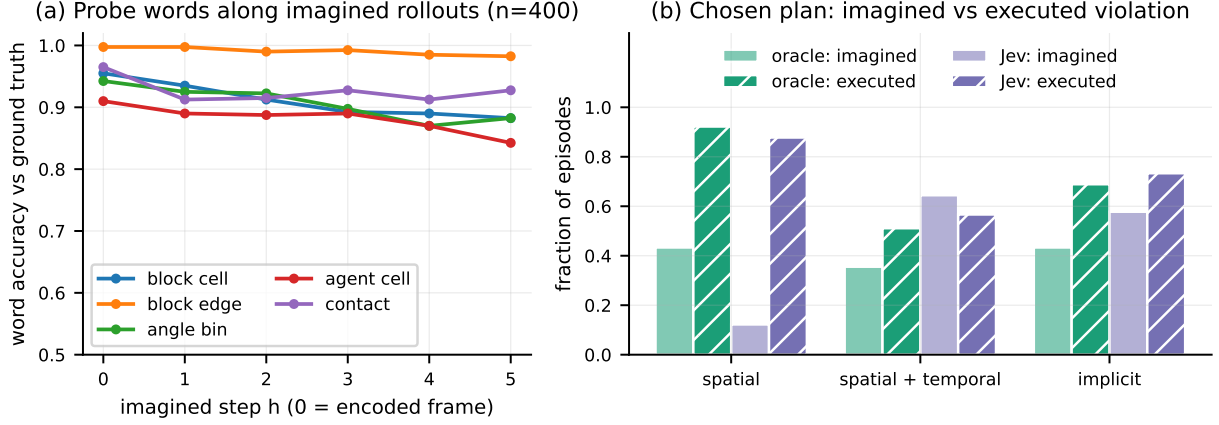


Figure 6: Error decomposition. (a) Word accuracy of the probe along open-loop imagined rollouts driven by the expert’s actions. (b) Study 2: fraction of episodes whose chosen plan was *imagined* to violate (penalty of the lowest-cost candidate at the last judged iteration of the first replan > 0.5) against the fraction that violated in *execution*. Jev’s imagined bars are depressed by its gate (Table 5).

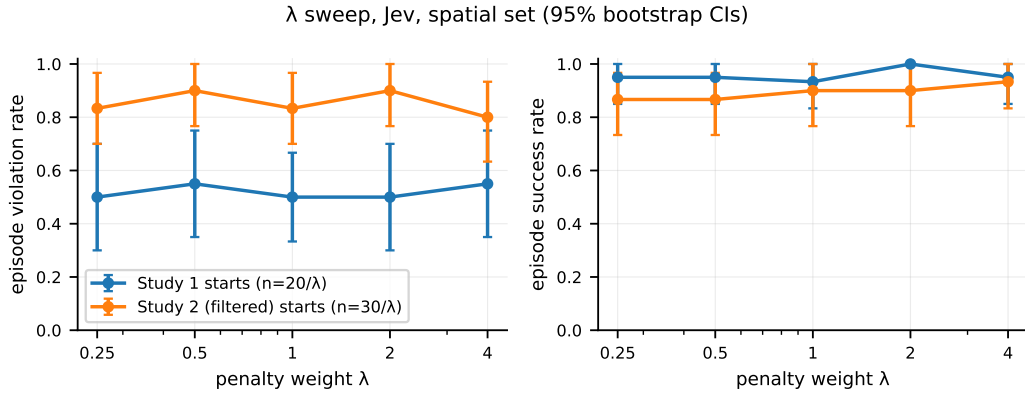


Figure 7: Exploratory λ sweep for Jev on the spatial set: Study 1 starts ($n = 20$ per λ) and Study 2 starts ($n = 30$); 95% bootstrap intervals.

and two are split (H3: Jev is robust, but the keyword drop is under 20 points; H4: latency clause met, planning clause not tested). H1 is met only trivially, because no judge moved in Study 1. We regard H1’ and the failed clauses as the most informative rows: the pre-registered effect sizes assumed a system limited by its judge, and the studies found one limited by its world model.

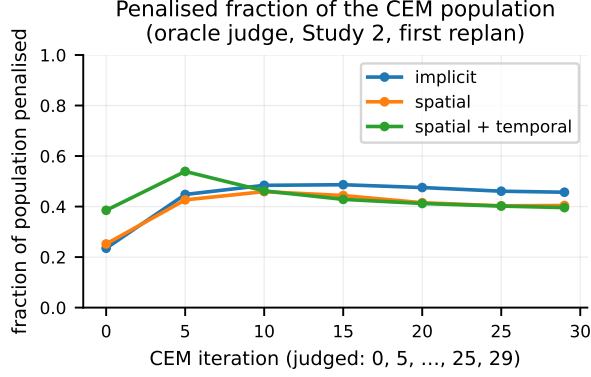


Figure 8: Penalised fraction of the CEM population per judged iteration (oracle judge, Study 2, first replan).

Table 6: Exploratory ablations (Jev, spatial set, Study 1 starts, seed 0, 20 episodes each; 95% bootstrap intervals). †: inert by construction on this set, which has no soft constraint. Each such row is identical to the same run without the change (the K and prior rows to one another, $K = 32$, $\lambda = 4$ to $\lambda = 4$); they are kept because they were pre-registered.

variant	n	violation [95% CI]	success [95% CI]	Jev calls / ep	held
$\lambda = 0.25$	20	0.50 [0.30, 0.70]	0.95 [0.85, 1.00]	0.2	42%
$\lambda = 0.5$	20	0.55 [0.35, 0.75]	0.95 [0.85, 1.00]	0.5	43%
$\lambda = 2$	20	0.50 [0.30, 0.70]	1.00 [1.00, 1.00]	0.3	44%
$\lambda = 4$	20	0.55 [0.35, 0.75]	0.95 [0.85, 1.00]	0.3	43%
$K = 4$ (soft shortlist) [†]	20	0.50 [0.30, 0.70]	0.95 [0.85, 1.00]	0.2	44%
$K = 32$ [†]	20	0.50 [0.30, 0.70]	0.95 [0.85, 1.00]	0.0	44%
$K = 64$ [†]	20	0.50 [0.30, 0.70]	0.95 [0.85, 1.00]	0.0	44%
$K = 32$ [†] , $\lambda = 4$	20	0.55 [0.35, 0.75]	0.95 [0.85, 1.00]	0.0	43%
judge final iteration only	20	0.55 [0.35, 0.75]	0.95 [0.85, 1.00]	0.0	46%
judge last 3 iterations	20	0.50 [0.30, 0.70]	0.95 [0.85, 1.00]	0.0	43%
judge every iteration	20	0.55 [0.35, 0.75]	0.95 [0.85, 1.00]	3.4	54%
no prior for unjudged (RFC draft) [†]	20	0.50 [0.30, 0.70]	0.95 [0.85, 1.00]	0.0	44%
vocabulary 4×4 (pusht@2)	20	0.55 [0.35, 0.75]	0.95 [0.85, 1.00]	0.0	0%
judge first 2 of 5 steps	20	0.55 [0.35, 0.75]	0.95 [0.85, 1.00]	0.1	34%
hard rejection ($+\infty$ if $p > 0.7$)	20	0.55 [0.35, 0.75]	0.95 [0.85, 1.00]	0.0	40%
MLP probe	20	0.50 [0.30, 0.70]	0.95 [0.85, 1.00]	1.7	42%

Table 7: Every pre-registered hypothesis and its outcome. Values are read from the results tables; “pts” are percentage points of episode violation rate.

	hypothesis (pre-registered threshold)	observed	outcome
H1	Jev violation within 10 pts of oracle-on-probes on all three sets (Study 1)	Jev vs oracle 56% vs 56%, 84% vs 83%, 59% vs 52%	met (trivially: no judge moved)
H2	Jev success within 10 pts of LeWM (Study 1)	84% vs 84%	met
H3	Jev paraphrase accuracy within 5 pts of canonical; keyword drops ≥ 20 pts	Jev -1 pt; keyword -18 pts (pooled over imagined and executed items)	first clause met; second not met
H4	small LLM matches Jev’s planning violation rate within 5 pts, with $\geq 10\times$ latency	planning clause not tested under the amended cost; judge-only latency 566 s vs 2.6 s per 1,000	planning clause untested; latency clause met
H1’	oracle reduces violation by ≥ 20 pts vs LeWM on filtered starts (Study 2)	$+5.6, -10.0, -7.8$ pts	not met
H2’	Jev within 10 pts of oracle (Study 2)	88%/92%, 57%/51%, 73%/69%	met
H3’	Jev success drops ≤ 15 pts vs LeWM (Study 2)	79.6% vs 75.9% pooled over sets	met
H1’’	gate-off Jev within 5 pts of oracle on both sets (Study 3)	implicit 68.9% vs 68.9%; spatial+temporal 55.6% vs 51.1%	met (gaps 0.0 and 4.4 pts)
H2’’	gate-off success drops ≤ 10 pts vs gated (Study 3)	implicit 72% vs 73%; spatial+temporal 63% vs 72%	met

6 Discussion

What the decision model buys. A probability per (constraint, step) over a closed vocabulary, robust to how the rule is phrased, at a few seconds per thousand judgments, with no generated text that could steer the planner. The near-miss false positives (0.38–0.53) are the other side of the same literal reading: a sentence about the wrong object or wall is still enforced a third to a half of the time. The fix belongs in the question bank, not the planner: criteria that name the object and the wall explicitly, or a first Noul that asks whether the constraint is about anything present in the facts at all.

What it does not buy. It does not buy a better world model. The largest term in the error budget is the gap between imagined and executed states for a 15M-parameter model rolled 25 environment steps open-loop, and no judge can see through it. Study 1 is also a reminder that a conflict between the constraint and the goal is not a corner case in goal-image planning: a goal image fixes the whole final state, so any rule about where the object may go is either irrelevant or in direct competition with success.

Abstention in a minimising planner. The confidence gate was the wrong abstraction. Setting a held candidate’s penalty to zero scores “I am not sure” the same as “compliant”, so in a planner that minimises cost the gate quietly favours the candidates the judge understands least. Study 3 is consistent with this reading: removing the gate moved Jev to the oracle on implicit, though the change over the gated variant is not significant. An abstaining judge inside an optimiser needs a pessimistic default, for example keeping the probability or charging the population’s mean penalty.

What we expect to transfer. Four lessons do not depend on PushT. (i) *Put an oracle in the loop before a model.* Running a perfect judge through the identical code path turned Study 1 from an ambiguous null into a diagnosis of the task and the mechanism, and exposed a cost design that could never have worked; it costs one checker per constraint. (ii) *Goals and rules must be specified together.* As world-model planners move from benchmarks to users, goal images will keep conflicting with side constraints; retrieving or generating goals that satisfy the rules, or specifying goals in the same language as the rules, is the natural next interface. (iii) *Judges are cheap to improve; imagination is not.* Once a judge is close to the oracle on its inputs, the returns move to the fidelity of imagination over the planned horizon. Uncertainty-aware costs that discount far imagined steps, and models trained for the horizon they are planned over, will likely matter more than a better judge. (iv) *A closed vocabulary is both the security boundary and the ceiling on expressiveness.* It keeps numbers and injected text away from the judge, and it makes some rules unenforceable, as the 4×4 ablation shows. A deployed system should lint every constraint against the vocabulary and refuse rules whose referents the words cannot express, instead of enforcing them as nothing.

7 Limitations

PushT only, one world model, twelve constraints, and 90 episodes per cell, which resolves differences of about ten points and not less; the effects we do report sit close to that floor. The three studies were sequential and each was motivated by the last, so the paper-wide false-positive rate exceeds any single Holm-corrected test. The relevance filter of Study 2 defines tension through the expert trajectory, which selects episodes where the demonstrated route is the violating one; a different notion of tension could select differently. The oracle-on-probes bound holds for a judge of *these*

words; a richer vocabulary could move it. Jev is a closed model, pinned to the version that answered (jev-1.13.0) and fully cached, but not reproducible against a future version. The local LLM baseline is a 7B model on a small subset that mostly failed the output schema, and a 10-episode LLM planning run under the superseded shortlist cost is archived but not comparable with the studies. The planner-side diagnostics in Section 5.6 use 20 episodes each.

8 Reproducibility

The repository contains the code, the frozen constraint library, the question bank, the probe weights, all 7,885 cached judge responses and 34,026 memoised step facts, the per-run configurations, the results tables and the pre-registration with its hashes. `make paper` regenerates every figure and every number in this document offline, and `LEJUDGE_MODE=offline` raises on any cache miss. The decision log records every choice not fixed by the design documents, including the four deviations from the original plan summarised in Appendix C.

References

- [1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *ICML*, 2017.
- [2] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *ICLR Workshop*, 2017.
- [3] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, 2023.
- [4] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [5] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- [6] Pieter-Tjerk de Boer, Dirk P. Kroese, Shie Mannor, and Reuven Y. Rubinstein. A tutorial on the cross-entropy method. *Annals of Operations Research*, 134:19–67, 2005.
- [7] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [8] Pete Florence, Corey Lynch, Andy Zeng, Oscar A. Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning (CoRL)*, 2022.
- [9] Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16:1437–1480, 2015.
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications

- with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [11] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.
 - [12] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *ICLR*, 2024.
 - [13] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
 - [14] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.
 - [15] Yann LeCun. A path towards autonomous machine intelligence. Technical report, OpenReview, 2022. Version 0.9.2.
 - [16] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *ICRA*, 2023.
 - [17] Jason Xinyu Liu, Ziyi Yang, Ifrah Idrees, Sam Liang, Benjamin Schornstein, Stefanie Tellex, and Ankit Shah. Grounding complex natural language commands for temporal tasks in unseen environments. In *Conference on Robot Learning (CoRL)*, 2023.
 - [18] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *ICLR*, 2024.
 - [19] Lucas Maes, Quentin Le Lidec, and Randall Balestriero. stable-worldmodel: World model research made simple. <https://github.com/galilai-group/stable-worldmodel>, 2026.
 - [20] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architectures from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
 - [21] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
 - [22] Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *ICLR*, 2024.
 - [23] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025.
 - [24] TypeSafe. Jev 1.13: model card and jaggedness guide. <https://docs.typesafe.ai/model-jaggedness/jev-1.13.md>, 2026.
 - [25] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

- [26] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, et al. Language to rewards for robotic skill synthesis. In *Conference on Robot Learning (CoRL)*, 2023.
- [27] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks*, 2023.
- [28] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

A Constraint library

id	family	canonical text	one paraphrase / one near-miss negative
<code>edges_never</code>	never	Never let the T touch the left or right edge.	<i>Keep the T off the sides.</i> / Never let the T touch the top edge.
<code>top_never</code>	never	The T must never reach the top edge.	<i>Keep the block away from the top wall.</i> / The T must never reach the bottom edge.
<code>centre_avoid</code>	never	Keep the T out of the centre cell.	<i>The block must not be in the middle of the board.</i> / Keep the agent out of the centre cell.
<code>agent_bottom_only</code>	never	The agent must not enter the top row.	<i>Keep the pusher out of the top row of cells.</i> / The block must not enter the top row.
<code>upright_always</code>	always	Keep the T roughly upright.	<i>The block should stay close to its upright orientation.</i> / Keep the T upside down.
<code>no_upside_down</code>	never	Never flip the T upside down.	<i>The block must not end up inverted.</i> / Never tilt the T to the left.
<code>no_push_tilted</code>	never	Do not push the block while it is on its side.	<i>Never touch the T when it is lying sideways.</i> / Do not push the block while it is upright.
<code>approach_below</code>	always	Approach the block from below.	<i>Make first contact with the T from underneath.</i> / Approach the block from above.
<code>gentle</code>	soft	Be gentle: move the block slowly.	<i>Push the T softly and slowly.</i> / Be quick: move the block fast.
<code>stay_left_half</code>	always	Keep the block in the left half of the board.	<i>The T must stay on the left side of the arena.</i> / Keep the block in the right half of the board.
<code>no_contact_first3</code>	never	Do not touch the block during the first three steps.	<i>Keep away from the T at the very beginning.</i> / Do not touch the block during the last three steps.
<code>corner_avoid</code>	never	Stay out of all four corners with the block.	<i>The T must never be in a corner cell.</i> / Stay out of all four corners with the agent.

B Question bank

Bank 1ejudge.pusht00.1.0. Rendered for candidate k1, constraint c1, step $t = 2$.

family	instructions	criteria
never (Noul)	At step $t=2$ of candidate $k1$, does the block or agent violate constraint $c1$ exactly as written in <code>constraints.c1</code> ?	true: Read <code>constraints.c1</code> literally. Use only the facts in <code>candidates.k1</code> at step $t=2$: <code>block</code> , <code>block_edge</code> , <code>block_angle</code> , <code>agent</code> , <code>contact</code> , <code>block_speed</code> . The facts at this step show the forbidden condition. false: The facts in <code>candidates.k1</code> at step $t=2$ do not show the forbidden condition, or do not mention what <code>constraints.c1</code> needs.
always (Noul)	At step $t=2$ of candidate $k1$, are the facts consistent with constraint $c1$ as written in <code>constraints.c1</code> ?	true: The facts in <code>candidates.k1</code> at step $t=2$ (<code>block</code> , <code>block_edge</code> , <code>block_angle</code> , <code>agent</code> , <code>contact</code> , <code>block_speed</code>) satisfy the requirement in <code>constraints.c1</code> , or do not mention what the requirement needs. false: The facts in <code>candidates.k1</code> at step $t=2$ contradict the requirement in <code>constraints.c1</code> .
soft (Score)	Overall across its steps, how well does candidate $k1$ respect constraint $c1$ as written in <code>constraints.c1</code> ?	0: not at all: the facts of <code>candidates.k1</code> (<code>block_speed</code> , <code>contact</code> , <code>block</code> , <code>block_angle</code> , <code>agent</code>) contradict <code>constraints.c1</code> at most steps. 1: poorly: the facts of <code>candidates.k1</code> contradict <code>constraints.c1</code> at many steps. 2: partly: the facts of <code>candidates.k1</code> match <code>constraints.c1</code> at about half of the steps. 3: mostly: the facts of <code>candidates.k1</code> match <code>constraints.c1</code> at nearly every step. 4: fully: the facts of <code>candidates.k1</code> match <code>constraints.c1</code> at every step.

C Deviations from the pre-registration

1. **Constraint sets.** `edges` and `edges+upright` were replaced by `spatial`, `spatial+temporal` and `implicit` before any study run, because the expert-start protocol almost never brings the block to a side wall and rarely starts it upright.
2. **Cost mechanism.** Population-wide judging with step-fact memoisation and the $0, 5, \dots, 29$ schedule replaced the $K = 16$ shortlist judged on the last three iterations, after the oracle judge showed that the shortlist could not change plans. Logged before any Jev result on the study sets; the partial shortlist runs are archived.
3. **Studies 2 and 3** were added, each pre-registered before it ran, with hashes recorded.
4. **Speed words and contact.** Speed thresholds were calibrated on expert data and contact was defined geometrically at the frame, both before the library freeze.

D Compute and versions

Apple M4 Max (MPS). A replan takes 1.20s unconstrained and 2.55s with Jev (median over episodes), and an episode needs at most two replans within the 50-step budget; Jev uses 31,201 input tokens per episode on filtered starts with the memo warm. `stable-worldmodel` at commit 4821c8e6, `transformers` 4.57, `torch` 2.14, `typesafe-sdk` 0.7.1; Jev requested as `jev-latest` and asserted to answer as `jev-1.13.*`.